O'REILLY®

4.1 Building Robust Tools with Validation and Logging

O'REILLY®

Beyond Basic Tool Creation

- Production tools require comprehensive validation, error handling, and observability
- Must be resilient, secure, and maintainable at scale
- Balance between functionality, performance, and reliability
- Consider the full lifecycle: development, deployment, monitoring, and evolution - independent of the agents using the tool

O'REILLY®

Input Validation and Type Safety

- Define explicit schemas for all tool parameters using type systems
- Validate data types, formats, ranges, and business rules
- Provide clear, actionable error messages for validation failures
- Prevent malformed data from reaching tool execution logic
- Consider optional vs required parameters with sensible defaults
- Validate dependencies between parameters (e.g., mutually exclusive options)

O'REILLY®

Error Handling Strategy

- Classify errors: validation errors, network errors, auth errors, resource errors
- Return structured error responses with error codes and context
  - Define clear error contracts for agent consumption
- Preserve error chains for debugging
- Implement graceful degradation when possible

O'REILLY®

Resilience and Retry Mechanisms

- Implement retry logic for network and external service failures
- Set appropriate timeout values for operations
- Configure maximum retry attempts and total retry duration
- Use exponential backoff with jitter
- Consider fallback strategies when retries are exhausted

O'REILLY®

Observability and Monitoring for Tools

- Log tool invocations with relevant parameters (sanitize sensitive data)
- Track execution time, success/failure rates, and error patterns
- Implement structured logging with correlation IDs for tracing (ex: a session_id to track tool calls across a user session/thread)
- Expose metrics: latency percentiles, throughput, error rates
- Include contextual metadata: user, session, request ID
- Set up alerting for anomalous behavior or degraded performance

O'REILLY®

Security Considerations

- Validate and sanitize all inputs to prevent injection attacks
- Implement proper authentication and authorization checks
- Use principle of least privilege for tool permissions
- Protect sensitive data in logs and error messages
- Implement and respect rate limiting to prevent abuse

O'REILLY®

Performance and Scalability

- Optimize for latency: minimize network calls, use caching strategically
- Implement connection pooling for database and API connections
- Set appropriate resource limits (ex: concurrent operations)
- Consider asynchronous execution for long-running operations

O'REILLY®

4.2 Building Multi-Agent Supervisor Systems

Introduction

- **Objective**
  - Learn how to design and implement a collaborative multi-agent system using LangGraph for financial analysis.
  - Explore the "divide-and-conquer" approach by creating specialized agents for distinct tasks or domains.

- **Core Idea**
  - Specialised agents to handle their own tasks
  - Router to determine which agent to call.

Multi-Agent Collaboration

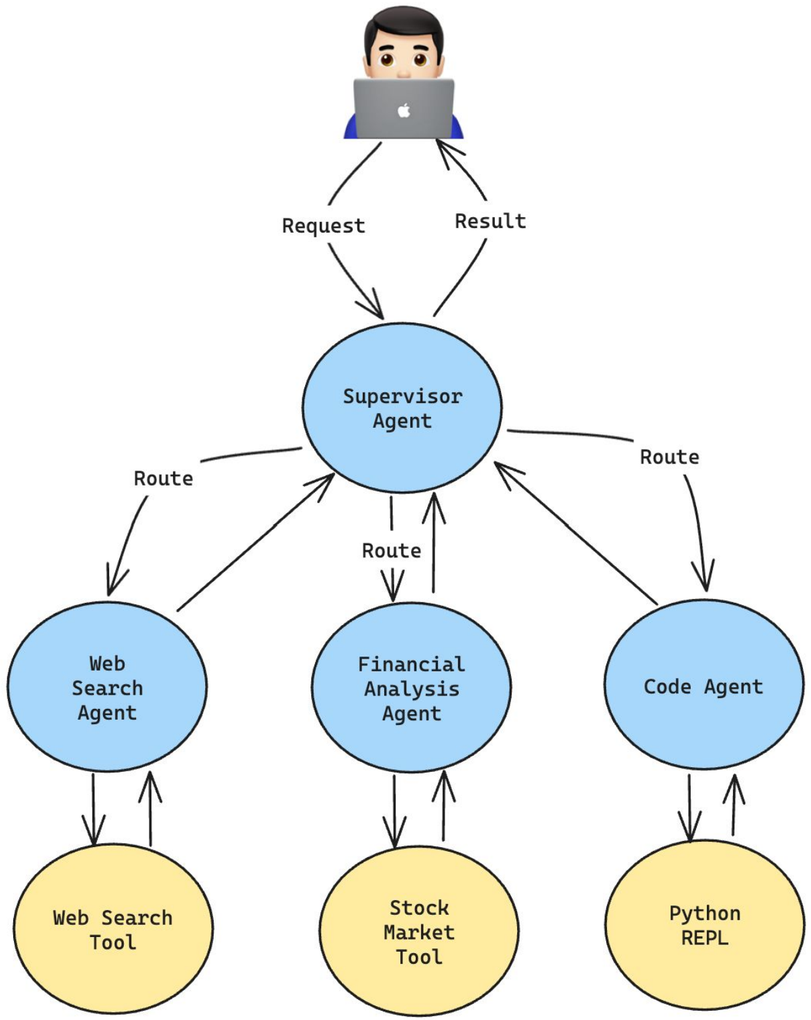
- **Challenges with Single Agents**
  - Limited effectiveness when handling numerous tools or complex, multi-domain tasks.
  - Even powerful models like GPT-4 can struggle with extensive tool usage.

- **Divide-and-Conquer Approach**
  - **Specialized Agents**: Create individual agents, each expert in a specific task or domain.
  - **Task Routing / Supervising**: Implement a supervisor agent that directs tasks to the appropriate agents.

Benefits of Multi-Agent Collaboration

- **Enhanced Efficiency**
  - Each agent focuses on its area of expertise, improving overall performance.
- **Scalability**
  - Easily add more specialized agents as needed without overcomplicating a single agent.
- **Robustness**
  - Reduces the likelihood of errors by compartmentalizing tasks.

Multi-Agent System: Financial Analysis



A Hierarchical Multi-Agent Workflow

O'REILLY®

4.3 Building Multi-Agent Swarm Systems

O'REILLY®

The Swarm Architecture

- **Decentralized Agent Collaboration**
  - Agents autonomously hand off tasks to peers
  - No central supervisor required
  - Dynamic, fluid collaboration patterns
- **Key Characteristics**
  - Peer-to-peer agent communication
  - Agents decide when to transfer control
  - Emergent collective behavior
  - Highly adaptable to changing requirements

O'REILLY®

Swarm vs Supervisor Architecture

Aspect	Swarm	Supervisor
Control	Decentralized	Centralized
Routing	Agent-initiated	Supervisor-directed
Scalability	Highly scalable	Limited by supervisor
Complexity	Emerges from interactions	Explicit in supervisor
Best For	Unpredictable agent selection, speed-critical, peer consultation	Known workflow steps, governance & control required

O'REILLY®

Core Concepts of Swarm Architecture

- Handoff Mechanisms
- Agent Specialization
- Network of Expertise

O'REILLY®

Handoff Mechanism

- Agents equipped with handoff tools to transfer control
- Each agent decides when another agent is better suited
- Handoff includes full context transfer
- Enables seamless task transitions

O'REILLY®

Agent Specialization

- Each agent has a specific domain of expertise
- Agents know their limitations and peer capabilities
- Clear handoff criteria defined in agent prompts
- Promotes modular, maintainable system design

O'REILLY®

Network of Expertise

- Forms a graph of interconnected specialists
- No single point of failure
- Knowledge distributed across the swarm
- Collective intelligence emerges from collaboration

O'REILLY®

Designing Effective Swarms

- **Defining Agent Roles**
  - Identify distinct domains of expertise
  - Ensure clear boundaries between agents
  - Avoid overlapping responsibilities
  - Consider typical task workflows
- **Establishing Handoff Rules**
  - When should an agent transfer control?
  - Which agent handles which type of query?
  - How to handle ambiguous cases?
  - Fallback strategies for edge cases

O'REILLY®

Dynamic Task Routing Process

1. User query arrives at default agent
2. Agent evaluates if task matches its expertise
3. If not optimal, agent selects appropriate peer
4. Context transferred via handoff tool
5. New agent continues from where previous left off
6. Process repeats until task completion

O'REILLY®

Dynamic Task Routing Example

1. Query: "I need a refund for order #12345, and can you help me find a replacement in Spanish?"
2. Triage agent receives request, identifies billing + language needs
3. Handoff to Billing agent with full context
4. Billing agent processes refund, then handoff to Sales agent
5. Sales agent finds replacement options, handoff to Spanish agent
6. Spanish agent translates response and delivers to user
7. Each agent autonomously decided when to pass control

O'REILLY®

Advantages of the Swarm Architecture

- No bottleneck from central coordinator
- Graceful degradation if individual agents fail
- Easy to add new specialized agents
- Scales horizontally with team size
- Flexible adaptation to new task types

O'REILLY®

4.4 Structuring Workflows with Subgraphs

O'REILLY®

Subgraphs in Agentic Workflows

- **Definition**
  - A subgraph is essentially a compiled graph that acts as a building block for complex agentic workflows
  - Think of subgraphs as complete, self-contained workflows that can be embedded as single nodes within larger graphs
- **Why Subgraphs Matter**
  - **Reusability**: Build module once, use in multiple workflows
  - **Easier testing**: Test components in isolation
  - **Improved maintainability**: Changes to one subgraph don't break others
  - **Clearer architecture**: Visual and logical separation of concerns

O'REILLY®

The Modularity Principle

- **Core Idea**
  - Break complex agentic systems into smaller, manageable pieces
  - Each subgraph solves one specific problem well
  - Parent graph orchestrates how subgraphs work together
  - Enables "separation of concerns" in workflow design
- **Benefits of Modular Design**
  - Teams can work on different subgraphs independently
  - Easier to reason about individual components
  - Swap out implementations without affecting the whole system
  - Scale complexity gradually as needs grow

O'REILLY®

State Management in Subgraphs

- **The State Contract Concept**
  - Parent graph maintains overall state schema
  - Subgraph defines its own state schema (typically a subset)
  - Parent state flows into subgraph, subgraph modifications flow back
- **State Scope and Isolation**
  - Build natural boundaries for what data each component accesses
  - Reduces cognitive load: developers only think about relevant state

O'REILLY®

Composing Workflows with Subgraphs

- **From Simple to Complex**
  - Start with basic linear graphs
  - Identify reusable patterns or phases
  - Extract those patterns into subgraphs
  - Compose subgraphs into sophisticated workflows

O'REILLY®

Composing Workflows with Subgraphs

- **Integration Patterns**
  - **Sequential:** Subgraphs execute one after another
  - **Conditional:** Router decides which subgraph to execute
  - **Parallel:** Multiple subgraphs process data simultaneously
  - **Nested:** Subgraphs can contain their own subgraphs (use sparingly)

O'REILLY®

Design Principles and Trade-offs

- **When to Use Subgraphs**
  - Workflow has natural conceptual boundaries
  - Need to reuse component across multiple applications
  - Want to test complex logic in isolation
  - Building hierarchical or team-based agent systems
  - Managing growing state complexity
- **When NOT to Use Subgraphs**
  - Simple, linear workflows with few steps
  - No reuse expected across different contexts
  - Adds unnecessary abstraction layer
  - Over-engineering simple problems

O'REILLY®

4.5 Implementing Parallel Task Execution in LangGraph

O'REILLY®

Static vs Dynamic Parallelism

- **Static Parallelism**
  - Fixed number of branches defined at graph design time
  - Use multiple edges from START to parallel nodes, then converge
  - Best when you always know exactly how many parallel tasks you need
- **Dynamic Paralellism**
  - Branch count determined at runtime based on input data
  - Example: process N search queries where N varies per request
  - Three approaches: Send API, AsyncIO, or Manual Threading

O'REILLY®

Using SendAPI

- Router node returns list of Send objects, each dispatching to a processor node
- LangGraph runs all dispatched nodes in parallel automatically
- Requires a reducer on state field to merge results from parallel branches
- Each parallel branch gets full graph features (state, checkpointing, etc.)
- Best for: complex per-item processing across multiple nodes

O'REILLY®

Using AsyncIO

- Use asyncio.gather() within a single node to run operations concurrently
- All parallel work stays contained in one node, returns aggregated results
- Lightweight - no extra nodes or state management overhead
- Best for: batching I/O-bound operations like API calls or database queries

O'REILLY®

Using Manual Threading

- ThreadPoolExecutor for I/O-bound, ProcessPoolExecutor for CPU-bound
- Explicit control over pool size, concurrency limits, and resource usage
- ProcessPoolExecutor bypasses Python's GIL for true CPU parallelism
- Best for: CPU-bound work, rate-limited APIs, or legacy sync code

O'REILLY®

Choosing Your Parallelism Approach

- Your approach will depend on your use case and there are multiple ways to implement parallelism within a node, at a node level or at a graph level
- Here's a handy reference to get you started
  - Does each item need its own node execution with state updates? Send API
  - Is it simple I/O that can be gathered in one place? AsyncIO
  - Is it CPU-bound or need thread pool control? Manual Threading

O'REILLY®

4.6 Comparing Sequential and Parallel Plan Execution

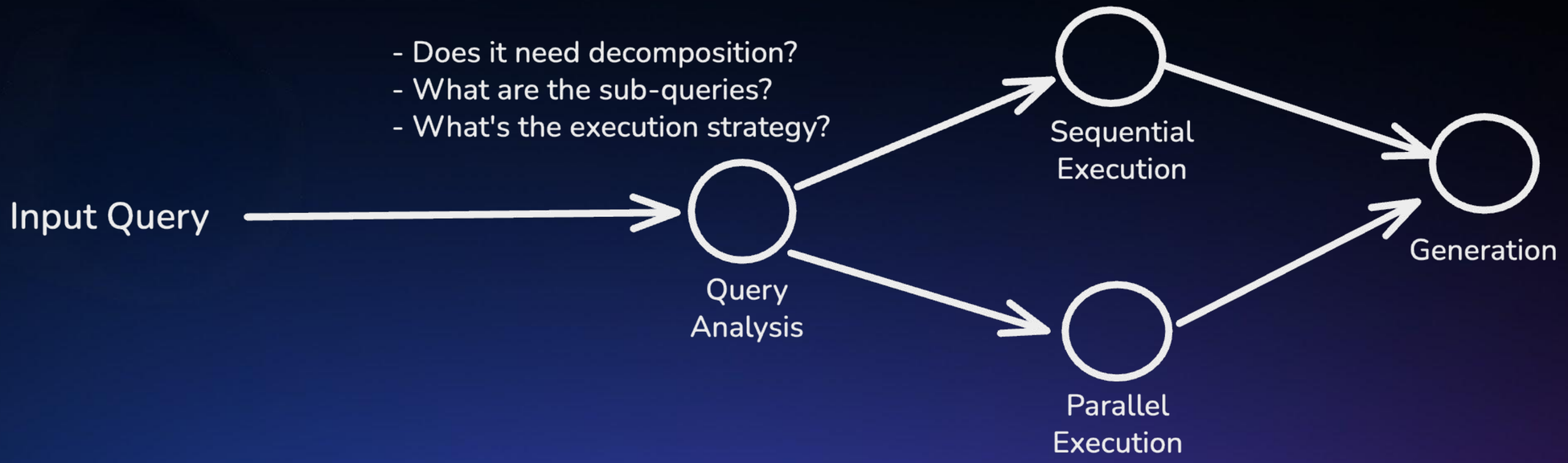
O'REILLY®

Query Analysis For Strategy Selection

- **First Step**: Use an LLM to analyze the query and determine if decomposition is needed
- **LLM Output**: Returns structured decision with sub-queries and recommended strategy

O'REILLY

Query Analysis For Strategy Selection



Typical Flow in Query Analysis + Execution

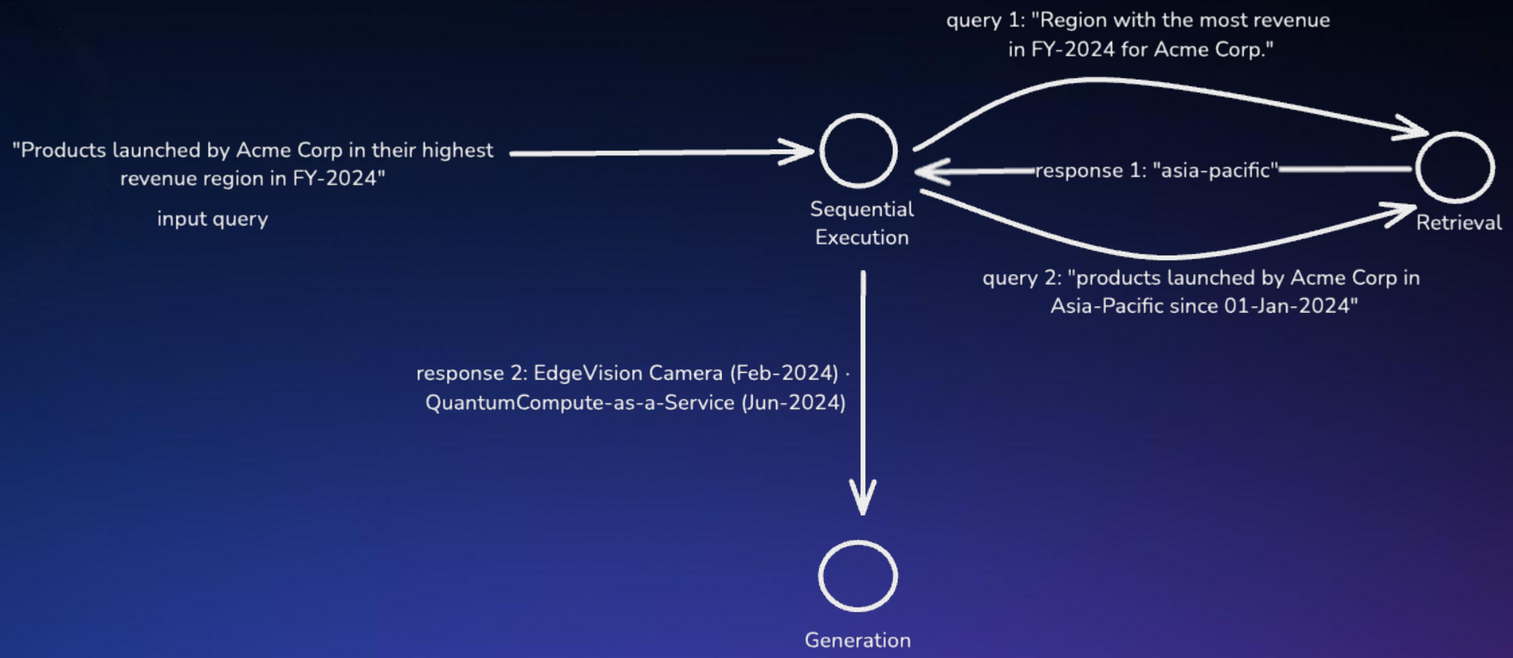
O'REILLY®

Sequential Execution

- **When to Use**: Sub-queries depend on previous results
- **Example Query**: "AI products launched by the company that acquired DeepMind in 2024"
- **Key Characteristic**: Each sub-query waits for the previous result before executing
- **Benefit**: Enables answers that require chaining information across multiple steps

O'REILLY®

Sequential Execution Example



An example of query decomposition with sequential execution

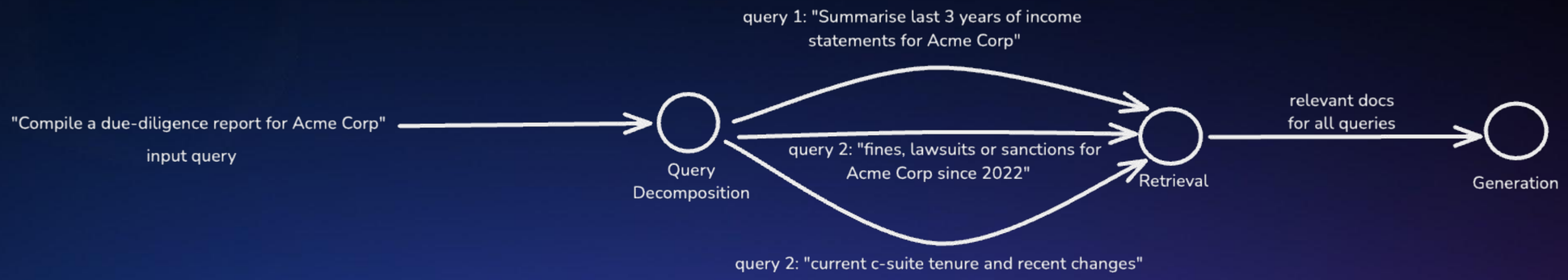
O'REILLY®

Parallel Execution

- **When to Use**: Sub-queries are independent from each other
- **Example Query**: "Summarize Tesla's Q4 2024 earnings, recent product launches, and leadership changes"
- **Key Characteristic**: All sub-queries execute simultaneously
- **Benefit**: Faster completion time (parallel I/O operations)

O'REILLY

Parallel Execution



An example of query decomposition with parallel execution

O'REILLY®

4.7 Developing Error Handling and Recovery Pathways

O'REILLY®

Error Categories in Agent Workflows

- **Transient:** Temporary network issues, rate limits (recoverable)
- **Permanent:** Invalid inputs, unauthorized access (not recoverable)
- **Partial:** Some tools succeed, others fail
- **Cascading:** One failure triggers downstream issues

O'REILLY®

Error Handling Flow

- **Detection → Decision → Action**
- **Detection:** Identify error type and severity at each step
- **Decision:** Determine appropriate recovery pathway
- **Action:** Route workflow to recovery node or alternative path

O'REILLY®

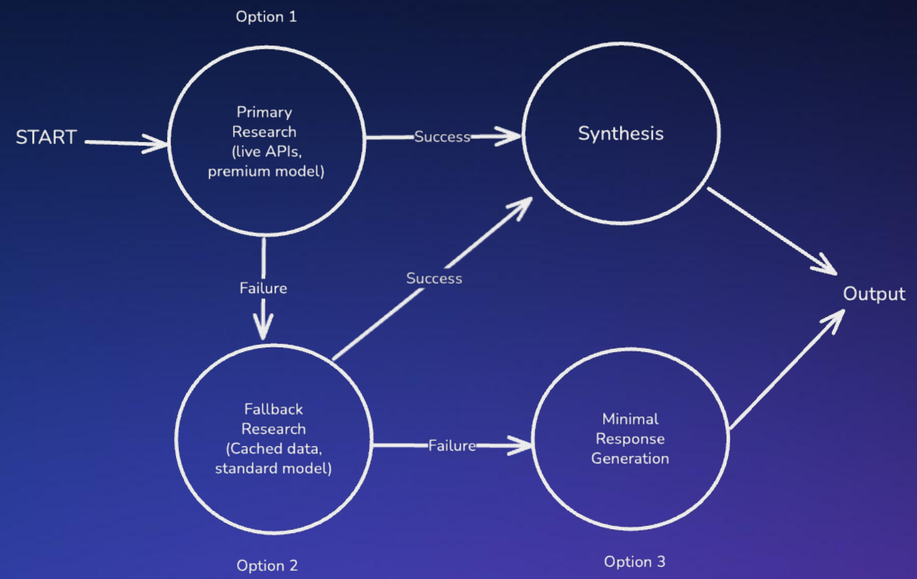
Error Recovery Pathways

- **Retry**: Attempt same operation again (transient errors)
- **Fallback**: Route to simpler alternative approach
- **Skip**: Continue workflow without failed component
- **Escalation**: Human-in-the-loop intervention
- **Abort**: Terminate gracefully with informative message

O'REILLY®

Workflow Level Fallback Strategy

- Design alternative execution paths across multiple nodes
- Each pathway uses different tools, models, or data sources
- Plan fallback pathways during workflow design phase



Fallback Strategy with 3 Options

O'REILLY®

Circuit Breakers

- **Purpose**: Prevent Cascading Failures
  - Monitor failure rate of external dependencies
  - Stop calling failing service to allow recovery time
  - Gradually restore service calls after timeout
- **Three Circuit States**
  - **Closed**: Normal operation, requests proceed to service
  - **Open**: Skip external call, immediately route to fallback
  - **Half-open**: Test recovery with limited requests

O'REILLY®

Circuit Breakers in Workflows

- **State Transitions**
  - Closed → Open: After N consecutive failures
  - Open → Half-open: After timeout period expires
  - Half-open → Closed: If test requests succeed
- **Workflow Integration**
  - Circuit state stored in workflow state or external store
  - Conditional routing checks circuit before calling service
  - Automatic fallback routing when circuit is open

O'REILLY®

Managing Error States

- Carry error information through workflow state
- Enable downstream nodes to make informed decisions
- Sample state to track errors:

```
state = {
    "input": original_input,
    "result": None,
    "error": {
        "type": "RateLimitError",
        "component": "search_tool",
        "retry_count": 2,
        "severity": "transient"
    },
    "fallback_level": 1  # Track degradation level
}
```

O'REILLY®

Common Pitfall: Silent Error Swallowing

Catch-all exception handlers that log and continue can mask serious issues.

The agent "works" but produces subtly wrong results.

Be explicit about which errors are recoverable and which should fail loudly.

O'REILLY®

4.8 Using Time Travel for State Branching

O'REILLY®

What is Time Travel?

- Time travel allows you to navigate through an agent's execution history like using Git version control.
- **It can be used to:**
  - **Understand reasoning:** Analyze steps that led to success
  - **Debug mistakes:** Identify where and why errors occurred
  - **Explore alternatives:** Test different paths to find better solutions

O'REILLY®

Time Travel Core Concepts

- **Checkpoints**
  - Automatic snapshots of graph state at each step
  - Contains: state values, metadata, next nodes
  - Organized by thread_id (conversation) and checkpoint_id (specific moments)
- **State History**
  - Complete timeline of all checkpoints
  - Accessible in reverse chronological order
  - Enables auditing and replay
- Braching
  - Fork execution from any checkpoint
  - Create alternate timelines
  - Original timeline remains intact

O'REILLY®

Time Travel Workflow in LangGraph

- **Step 1: Run with Checkpointing Enabled**
  - Compile graph with a checkpointer (e.g., MemorySaver)
  - Execute with thread_id to track conversation

- **Step 2: Identify Target Checkpoint**
  - Review execution history using get_state_history()
  - Find checkpoint before error or decision point
  - States returned in reverse chronological order (newest first)

O'REILLY®

Time Travel Workflow in LangGraph

- **Step 3: Update State (Optional)**
  - Modify graph state at the checkpoint
  - Inject corrections or alternative inputs
  - Creates new branch automatically
- **Step 4: Resume Execution**
  - Continue from checkpoint with invoke(None, config)
  - Passing None signals "continue from checkpoint"
  - New branch executes with modified state

O'REILLY®

Use Cases

- **Interactive Debugging**
  - Navigate back to decision point when agent produces wrong answer
  - Modify reasoning and see corrected execution path
- **Human-in-the-Loop Corrections**
  - User reviews and edits agent's proposed action before execution
  - Original attempt preserved for analysis

O'REILLY®

Use Cases

- **A/B Testing Agent Decisions**
  - Create multiple branches with different parameters from decision point
  - Compare outcomes side-by-side to select best approach
- **Post-Mortem Analysis**
  - Replay full execution history when production agent fails
  - Test fixes before deploying

O'REILLY®

4.9 Optimizing Prompts and Tool Selection

O'REILLY®

Why Prompt Quality Matters

- Prompts directly influence agent behavior
- Poor prompts lead to hallucinations
- Unclear instructions cause wrong tool selection
- Optimization reduces errors and costs
- **Key Optimization Goals**
  - Improve reasoning accuracy
  - Reduce hallucinations
  - Better tool selection
  - Efficient token usage

O'REILLY®

The Right Altitude Principle

- **Avoid extremes**: too specific (brittle) vs. too vague (ineffective)
- **Balance**: Strong heuristics that guide without over-prescribing
- **Structure**: Organize prompts into clear sections (background, instructions, tools, output format)
- Start minimal, add instructions only when specific failure modes appear

O'REILLY®

Prompt Optimization for Tool Selection

- **Explicit Guidelines for Tool Selection**
  - Clear tool descriptions from agent's perspective
  - Decision criteria and sequencing logic for when to use each tool
  - Frameworks like LangChain inject tool docstrings into main prompt

O'REILLY®

Prompt Optimization for Tool Selection

- **Three Critical Agent Reminders**
  - **Persistence:** "Keep going until task is completely resolved"
    - Prevents premature stopping (~20% performance improvement)
  - **Tool Usage Over Guessing:** "If unsure, use tools - do NOT guess"
    - Dramatically reduces hallucinations
  - **Plan and Reflect:** Think before acting, verify outputs make sense
    - Additional 4% performance gain

O'REILLY®

Using LLM's Built-in Reasoning Capabilities

- **Understanding Reasoning Models**
  - Reasoning models use additional "thinking" tokens to work through problems step-by-step before responding, unlike standard models that generate responses directly
  - These models handle complex reasoning internally without explicit prompting
- Examples
  - **GPT-5/5.1**: Reasoning model with API toggle to enable / disable reasoning (or thinking) mode
  - **Claude 4+** can use extra tokens to enable extended thinking
  - Most new foundational LLMs have a thinking mode

O'REILLY®

When to Use Reasoning vs Standard Models

- **Reasoning Models**
  - Enable for complex problems requiring multi-step analysis
  - Mathematical proofs and calculations
  - Strategic planning and decision-making
  - Complex coding challenges with multiple dependencies
  - Scientific problem-solving requiring deep analysis
- **Standard Models**
  - Use for everyday tasks
  - General queries and conversations
  - Simple tool usage and data retrieval
  - Fast responses needed
  - Cost-sensitive applications

O'REILLY®

Adding Few-Shot Examples for Consistency

- **When to Use Few-Shot Examples**
  - Demonstrate proper tool usage sequences and output formatting
  - Teach domain-specific behaviors not in common knowledge
  - Establish consistent response patterns
- **Quality Over Quantity**
  - Use 2-3 well-chosen diverse examples, so that ach example should demonstrate a distinct pattern
  - Show both simple and complex cases

O'REILLY®

Caveat: Reasoning Models May Overfit on Few-Shot

- Recent research suggests that reasoning models (GPT-5, Claude 4.5 Sonnet/Haiku Extended Thinking) can follow examples too rigidly
- Your mileage may vary, so only add few shot examples when needed, and test/version prompt changes vigorously

O'REILLY®

Continuous Improvement Philosophy

- Prompt optimization is ongoing, not one-time
- New failure modes emerge with different inputs and use cases
- Models improve over time; re-evaluate periodically
- Balance prompt complexity with maintainability
- Document what works and why for future reference

O'REILLY®

Common Pitfall: Optimization Without Measurement

- A common pattern: developer tweaks prompt, runs 2-3 test queries, declares it "better," and deploys.
- Two weeks later, support tickets reveal the change broke 15% of queries.
- Don't optimize prompts based on gut feel.
- Even a small golden dataset of 20 representative queries will catch regressions that informal testing misses.

O'REILLY®

4.10 Managing Context Windows Effectively

O'REILLY®

What is a Context Windows?

- The maximum amount of text (in tokens) an LLM can process at once
- Includes: system prompt + conversation history + tool outputs + current input

O'REILLY®

Why Manage the Context Window

- **Hard limit**: Exceeding it causes errors or automatic truncation
- **Soft limit**: Long contexts degrade LLM performance even within limits
- **Quality degradation**: LLMs get "distracted" by irrelevant or stale information
- **Cost & speed**: Longer context = higher API costs + slower responses
- **Core Principle**
  - Feed only relevant context to the LLM
  - More context != better results

O'REILLY®

Why Context Quality Matters

- More information means more "distractions"
- Stale conversation turns reduce focus on current task
- Off-topic messages dilute the agent's understanding
- The "Lost in the Middle" Problem:
  - LLMs struggle to effectively use information positioned in the middle of long input contexts

O'REILLY®

Context Quality Example

- Customer has 40-turn conversation about returning a laptop
- Return completed successfully at turn 35
- Customer asks: "What's your warranty policy on headphones?"
- Agent doesn't need: laptop model, return shipping details, refund confirmation
- Relevant context: Only the new question + product knowledge tools

O'REILLY®

Context Management Strategies Overview

- **Five Main Strategies**
  - **Truncation** - Keep only N most recent messages
  - **Token-Aware Truncation** - Count actual tokens, stay within limits
  - **Deletion** - Selectively remove specific messages
  - **Summarization** - Compress old messages into summaries
  - **Intelligent Pruning** - Score and keep most important messages
- **Key Insight**: Choose based on your use case complexity and context preservation needs

O'REILLY®

Truncation

- **How It Works**
  - Keep only the N most recent messages
  - Always preserve system message + recent conversation
- **Implementation**
  - Set a fixed message count (e.g., keep last 10 messages)
  - Drop oldest messages when limit exceeded
- **Trade-off:** Simple but loses older context entirely
- **Best for:** Short sessions, simple tasks, chatbots without long-term memory needs

O'REILLY®

Token-Aware Truncation

- **How It Works**
  - Count actual tokens, not just message count
  - Stay within exact model limits
- **Why Tokens Matter**
  - "Hello" = 1 token, but a code block might be 500 tokens
  - Message count is unreliable for context size
- **Implementation**
  - Use tokenizer (e.g., tiktoken) to count tokens per message
  - Remove oldest messages until under token budget
- **Trade-off:** More accurate sizing but still loses context
- **Best for:** Cost-sensitive applications, precise budget control

O'REILLY®

Context Deletion

- **How It Works**
  - Remove specific messages from history based on criteria
  - Target stale or irrelevant turns
- **What to Delete**
  - Completed sub-tasks with no future relevance
  - Off-topic conversation branches
  - Repetitive confirmations or acknowledgments
- **Trade-off:** Requires logic to identify what to remove, which can be complex
- **Best for:** Long sessions with topic shifts, multi-task conversations

O'REILLY®

Context Summarization

- **How It Works**
  - Compress old messages into a summary
- **Example**
  - Before: [msg1, msg2, ... msg50, msg51, msg52]
  - After: ["Summary of msg1-50", msg51, msg52]
- **Implementation**
  - Trigger summarization at threshold (e.g., every 20 messages)
  - Use LLM to generate summary of older messages
- **Trade-off:** Costs an extra LLM call, may lose nuance
- **Best for:** Long conversations where history matters

O'REILLY®

Intelligent Pruning

- **How It Work:**
  - Score messages by importance, keep highest-scoring messages, drop the rest
- **Scoring Criteria**
  - **Always preserve:** System message, first user message, recent messages
  - **Score middle messages by:** tool calls, keywords, length, relevance
- **Implementation**
  - Assign importance scores to each message
  - Sort by score, keep top N or top X% of tokens
- **Trade-off:** Complex logic, computational overhead
- **Best for:** Mission-critical applications, agents where context quality is paramount

O'REILLY®

Common Pitfall: Trusting the Large Context Window

Models advertise 200K or 1M context windows, so developers assume they can dump everything in.

Reality: performance degrades long before you hit the limit. The "lost in the middle" problem means information in the center of long contexts gets ignored.

A 128K context window doesn't mean you should use 128K tokens. Treat large windows as safety margin, not a target.

O'REILLY®

4.11 Testing and Evaluating AI Agents

O'REILLY®

Mindset Mismatch in Testing AI Agents

- According to latest research:
  - 70%+ effort on deterministic components (tools, workflows) vs. <5% on LLM core
  - Prompts tested in only ~1% of cases despite being critical entry points
  - **Mismatch**: effort goes to low-risk areas, ignores high-risk LLM reasoning
  - Traditional methods dominate; purpose-built agent testing patterns see <1% adoption

O'REILLY®

Beyond Traditional Testing Paradigms

- **Evaluation-Driven Development**
  - Test-Driven Development (TDD) assumes deterministic systems
  - Evaluation-Driven Development (EDD) for non-deterministic agents
  - Continuous evaluation throughout development lifecycle
  - Real-time feedback loops inform agent improvements

O'REILLY®

Three Pillars of EDD

- **Pre-Deployment Evaluation**
  - Golden dataset construction
  - Multi-dimensional benchmarking
  - Adversarial testing
- **Real-Time Monitoring**
  - Agent trajectory tracking
  - Intermediate step validation
  - Cost and latency monitoring
- **Post-Deployment Analysis**
  - User feedback integration
  - Behavioral drift detection
  - Continuous benchmark updates

O'REILLY®

Output Quality Dimensions

- **Task Completion**: Did agent achieve the goal?
  - Success Rate (SR) as primary metric
  - Partial completion scoring for complex tasks
- **Semantic Correctness**: Is the answer meaningful?
  - Semantic similarity to reference answers
  - LLM-as-a-Judge for nuanced evaluation
- **Faithfulness**: Grounded in provided context?
  - Fact verification against knowledge base
  - Hallucination detection

O'REILLY®

Process Quality Dimensions

- **Planning Effectiveness:** Did agent explore appropriately?
  - Reward effective information gathering
  - Penalize unnecessary tool calls
  - Measure decision tree efficiency
- **Tool Usage:** Right tools, right sequence?
  - Tool selection accuracy
  - Tool call parameter correctness
  - Multi-step coordination patterns
- **Reasoning Transparency:** Can we trace the logic?
  - Intermediate step coherence
  - Explanation quality

O'REILLY®

Operational Dimensions

- **Latency:** Response time within acceptable bounds
- **Cost:** Token usage and API calls optimized
- **Safety:** No harmful, biased, or toxic outputs
- **Robustness:** Consistent under adversarial inputs

O'REILLY®

Evaluation Methods for Agent Testing

Method	Use Case	Strictness	Requires Reference Dataset?
Semantic Similarity	General response equivalence	Medium	Yes
LLM-as-a-judge	Nuanced quality assessment	Flexible	Optional - can work either way
Pattern Matching	Prohibited content detection	Strict	No - checks against rules
Classification Metrics	Intent detection/routing accuracy	Strict	Yes - needs labeled data
Human Evaluation	Ground truth for ambiguous cases	Gold Standard	Optional

O'REILLY®

What is a Golden Dataset?

- **Core Concept**
  - Curated collection of test cases with known expected behaviors
  - Reference standard for evaluating agent performance
  - Living benchmark that evolves with your system
- **What Makes a Golden Dataset**
  - Representative sample of real user interactions
  - Edge cases that historically caused failures
  - Adversarial examples testing safety boundaries
  - Continuously updated from production incidents

O'REILLY®

Best Practices for Constructing a Golden Dataset

- Start small (10-20 cases), grow incrementally
- Version datasets alongside agent versions
- Multiple acceptable outputs per input
- Annotate reasoning traces in addition to final answers
- Mine production logs for real user queries
- Include expected tool usage patterns
- Add difficulty/complexity labels

O'REILLY®

Agent-as-a-Judge

- Use specialized agent to evaluate another agent
- Examines entire decision chain, not just final output
- Can assess reasoning quality, tool selection rationale
- Scales better than human evaluation

O'REILLY®

Judge Agent Design Principles

- Clear evaluation rubrics in system prompt
- Access to full agent trajectory (messages, tools, state)
- Structured output format (scores + justifications)
- Calibrated against human judgments initially

O'REILLY®

Agent-as-a-Judge Limitations

Limitation	Description	Mitigation
Self-Enhancement Bias	Same model family as judge tends to rate itself higher	Use different model families for agent and judge
Position Bias	Judges favor first or last items in comparison lists	Randomize order, run multiple evaluations
Verbosity Bias	Longer responses often rated higher regardless of quality	Include length-normalization in rubrics
Hallucinated Reasoning	Judge may invent justifications for scores	Require citation of specific evidence from trajectory
Inconsistent Scoring	Same input yields different scores across runs	Use low temperature, aggregate multiple judgments
Domain Blindness	Judges lack domain expertise for specialized tasks	Combine with domain-specific deterministic checks

O'REILLY®

Common Pitfalls

- **LLM-as-Judge Without Calibration**
  - Your judge LLM might have different standards than your users.
  - It marks outputs as "good" that users hate, or vice versa.
  - Calibrate judge prompts against human evaluations before trusting automated scores.
- **Golden Dataset Staleness**
  - Your golden dataset reflects queries from 6 months ago.
  - User behavior has shifted, but your tests still pass.
  - Regularly refresh golden datasets with recent production queries to catch drift.

O'REILLY®

4.12 Fine-Tuning Agents with Feedback and Monitoring

O'REILLY®

Production Feedback for Improvement

- **Feedback Signals to Collect**
  - User satisfaction ratings and explicit comments
  - Quality regression alerts from automated evaluation
  - Edge cases discovered in real-world usage
- **How to Analyze for Improvements**
  - Track satisfaction trends over time (not individual scores)
  - Identify common failure patterns by query category
  - Monitor behavioral drift (response quality degradation)

O'REILLY®

Human Driven Feedback Analysis

- Review low-rated interactions with user comments
- Identify common critique themes (tone, accuracy, helpfulness)
- Extract exact user phrasing for prompt examples
- Compare failed vs. successful responses for same query types

O'REILLY®

Automated Improvement Strategies

LLM-Assisted Improvement Generation

- Use meta-LLM to analyze failure clusters and propose prompt fixes
- Generate multiple prompt variations addressing identified issues
- Automatically create A/B test candidates from feedback patterns
- Synthesize common user critiques into actionable prompt changes

O'REILLY®

Automated Improvement Strategies

A/B Testing Workflow

- Define experiment: Control (current) vs. Treatment (improved) prompt
- Configure traffic split (e.g., 90/10 for safe testing)
- Set success metrics: Primary (satisfaction) + guardrails (latency, cost)
- Auto-rollback if treatment underperforms or causes regressions
- Promote winner when statistical significance reached (e.g., $p < 0.05$)

O'REILLY®

Mapping Issue Types to Fix Strategies

- Prompt clarity → Refine instructions/add examples
- Tool selection errors → Improve tool descriptions
- Tone/style mismatches → Adjust persona definition
- Knowledge gaps → Update retrieval/context sources
- Reasoning failures → Restructure thinking steps/ use more capable model

O'REILLY®

Prioritizing Impact

- Distinguish prompt fixes (fast) vs. architecture changes (slow)
- Create test cases from production failures
- Sequence: Quick wins first, then systemic improvements

O'REILLY®

Best Practices for Production Fine-Tuning

- **Version every change:** Tag each prompt/config version in monitoring to correlate feedback with specific iterations.
- **Test before deploying**: Validate improvements against golden datasets (4.13) and production failure cases to prevent regressions.
- **Start small, scale cautiously**: Use low-traffic A/B tests (5-10%) with auto-rollback before full deployment.
- **Balance automation with oversight**: Automate analysis and testing, but require human approval for production changes.
- **Close the feedback loop**: Feed production failures back into golden datasets and pre-deployment benchmarks to strengthen future evaluations.

O'REILLY®

4.13 Designing APIs for Long-Running Agent Tasks

O'REILLY®

Why Agents Need Long-Running API Patterns

- The Problem with Synchronous HTTP
  - Standard request/response expects fast replies (seconds)
  - Agent tasks can take minutes to hours (research, multi-step reasoning, tools)
  - HTTP timeouts, load balancer limits, and client disconnects break synchronous flows
- Agentic Task Characteristics
  - **Unpredictable duration**: LLM calls, tool execution, retries all add latency
  - **Multi-step workflows**: agents iterate through reasoning loops
  - **External dependencies**: web searches, API calls, file processing

O'REILLY®

Patterns for Long-Running Tasks

- **Polling**: Client periodically checks status endpoint
  - Best for simple clients, broad compatibility
- **Webhooks**: External service calls your server on events
  - Best for event-driven triggers (server-to-server only)
- **Streaming/Webhooks**: Persistent connection with incremental updates
  - Best for real-time UIs, interactive experiences
- **Key Decision**: Who initiates the update check?
  - **Polling**: Client pulls
  - **Webhooks/Streaming**: Server pushes

O'REILLY®

Polling Pattern Implementation

- **Submit Task → Get Task ID → Poll for Status**

```
# 1. Client submits task
POST /tasks → {"task_id": "abc123"}

# 2. Client polls status
GET /tasks/abc123 → {"status": "running", "progress": 45}
GET /tasks/abc123 → {"status": "completed", "result": {...}}
```

O'REILLY®

Polling: Design Considerations

- **Server Side**
  - Return task ID immediately (don't block on agent execution)
  - Include progress indicators when possible (steps completed, percentage)
  - Store task state in persistent storage (Redis, database)
- **Client Side**
  - Set appropriate poll intervals (too fast = wasted resources, too slow = poor UX)

O'REILLY®

Webhooks for Event-Driven Agents

- **What is a Webhook?**
  - An HTTP callback: a server sends a POST request to another when an event occurs
  - "Don't call us, we'll call you" - instead of polling, you get notified
  - You register a URL with a service, and it calls that URL when something happens
- **When to Use Webhooks**
  - Responding to external events (Slack messages, GitHub PRs, Stripe payments)
  - Integrating with third-party services that support webhook notifications
  - Building event-driven architectures where your agent reacts to triggers

O'REILLY®

Streaming Responses from Server

Persistent Connection with Server-Sent Events (SSE):

```
# Client connects to streaming endpoint
GET /tasks/abc123/stream

# Server sends incremental updates
data: {"type": "progress", "step": "searching", "detail": "..."}
data: {"type": "progress", "step": "analyzing", "detail": "..."}
data: {"type": "result", "content": "Final answer..."}
```

O'REILLY®

Streaming Responses from Server

Design Considerations

- Use SSE for simple one-way updates, WebSockets for bidirectional
- You can stream both event updates, and response tokens, and handle them in the client appropriately
- Handle client reconnection gracefully (resume from last event)
- Stream partial results as agent generates them (token streaming)
- Set appropriate timeouts and keepalive pings

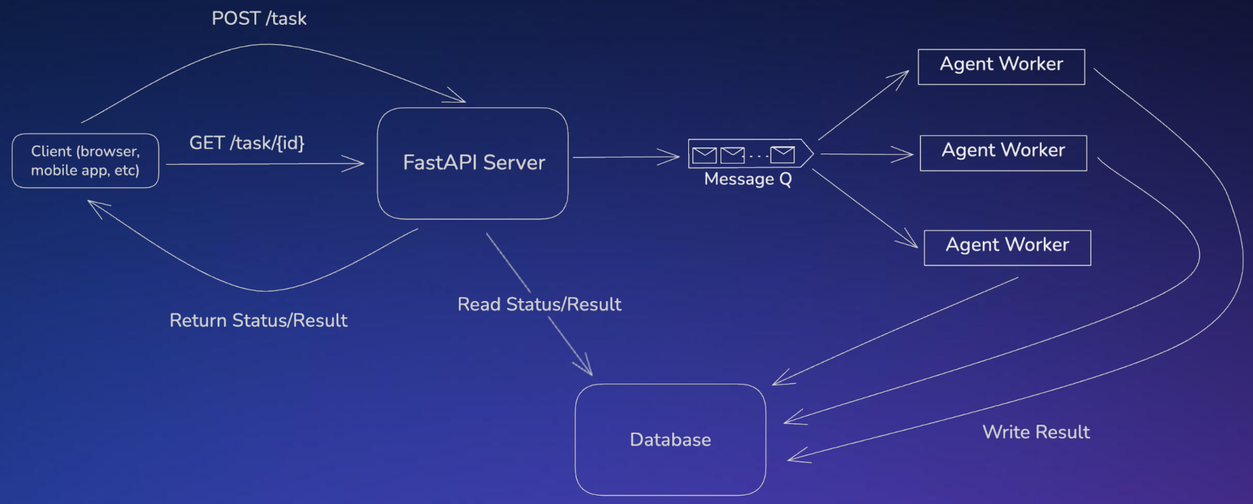
O'REILLY®

4.14 Deploying Agents in Worker Node Architectures

O'REILLY®

Why Worker Node Architecture for Agents?

- **What is Worker Node Architecture?**
  - A pattern that separates request handling from task execution
  - Web servers accept requests and queue them; worker processes execute them
  - Workers are independent processes that pull and process jobs asynchronously



A Worker Node Architecture with Polling

O'REILLY®

Why Worker Node Architecture for Agents?

- **The Problem with Running Agents in Web Requests**
  - Agent tasks are slow (seconds to minutes) and unpredictable
  - Web servers have request timeouts and limited concurrent connections
  - One slow agent task blocks resources for other users
  - Server restarts or deploys kill in-flight agent work
- **Worker Architecture Solves This**
  - Decouple task submission from task execution
  - Web server stays responsive (returns immediately with job ID)
  - Workers run agents independently, can retry on failure
  - Scale workers separately from web servers

O'REILLY®

FastAPI + Worker Integration Sample

```python
@app.post("/agent/run")
async def run_agent(request: AgentRequest):
    job_id = str(uuid.uuid4())
    # Enqueue job, return immediately
    queue.enqueue(run_agent_task, job_id, request.query)
    return {"job_id": job_id, "status": "queued"}

@app.get("/agent/status/{job_id}")
async def get_status(job_id: str):
    result = result_store.get(job_id)
    return {"job_id": job_id, "status": result.status, "output":
result.output}
```

O'REILLY®

Worker Implementation Example

- **Worker Responsibilities**
  - Pull jobs from queue (automatic with Celery/RQ)
  - Update status as job progresses
  - Handle timeouts and retries
  - Store results for later retrieval

```python
def run_agent_task(job_id: str, query: str):
    result_store.set(job_id, status="running")
    try:
        # Run the actual agent (this is the slow part)
        output = agent.invoke({"input": query})
        result_store.set(job_id, status="completed", output=output)
    except Exception as e:
        result_store.set(job_id, status="failed", error=str(e))
```

O'REILLY®

Python Worker Frameworks

- **Celery:** Uses Redis, RabbitMQ, or SQS; best for production systems and complex workflows
- **RQ (Redis Queue):** Uses Redis only; best for simple setups, quick to learn
- **Dramatiq:** Uses Redis or RabbitMQ; best for modern API with good defaults
- **Huey:** Uses Redis or SQLite; best for lightweight setups with minimal dependencies

O'REILLY®

4.15 Scaling Agents for Production Environments

O'REILLY®

Core Scaling Considerations

- **Horizontal Scaling:** Most components can be scaled horizontally since core agent operations are I/O bound
- **Statelessness**: Externalize all state so any worker can handle any request
- **Load Balancing**: Distribute requests and provide redundancy
- **Message Queues**: Buffer work and control concurrency
- **State Store Scaling**: Handle growing conversation history and checkpoints
- **Cost Control**: Manage spend across users and tenants

O'REILLY®

Horizontal vs Vertical Scaling

- **Vertical Scaling (Scale Up)**
  - Add more power to existing machines (CPU, RAM, disk)
  - Simpler to implement since no distributed coordination needed
  - Hard ceiling: eventually you hit hardware limits
  - Single point of failure: one server down means service down
- **Horizontal Scaling (Scale Out)**
  - Add more machines to distribute the workload
  - No theoretical ceiling—keep adding instances as needed
  - Requires stateless design and external state management
  - Built-in redundancy: one server down, others continue

O'REILLY®

Why Agents Demand Horizontal Scaling

- **The Nature of Agent Workloads**
  - High Latency: LLM inference and tool execution take seconds to minutes.
  - Blocking Operations: A single fast CPU cannot force an LLM to generate tokens faster.
  - **The Solution**: You don't need a faster machine; you need more machines running in parallel.
- **The Architecture of Resiliency**
  - **Stateless Workers**: Any worker can handle any agent task.
  - **Decoupled State**: Context and memory live in shared storage (Redis/Postgres), not on the server.
  - **Fault Tolerance**: If a worker crashes mid-thought, the job is simply re-queued to another node.

O'REILLY®

The Stateless Agent Architecture

- **Statelessness:** Workers store no data locally; they fetch context from a DB, act, and save it back.
- **The Constraint:** Agents accumulate massive state (history, tool outputs) unlike simple web requests.
- **The Anti-Pattern:** "Sticky sessions" lock users to specific servers, preventing failovers
- **The Solution:** Externalize all state. Workers fetch context, act, and save back.
- **Business State:** Persist critical data (IDs, final results) in durable DBs (Postgres).
- **Execution State:** Offload volatile reasoning/context to fast KV stores (Redis).
- **Outcome:** Any worker handles any step; pause/resume becomes trivial.

O'REILLY®

Load Balancing and the API Tier

- **Mechanism:** A reverse proxy that distributes incoming requests across a pool of available servers.
- **Scalability Contribution:** Decouples clients from specific servers, allowing addition or removal backend nodes dynamically.
- **Zero-Downtime:** Reroutes traffic away from updating nodes, ensuring users never experience outages during deployments.
- **Connection Management:** Maintains the long-lived WebSocket/SSE connections required for streaming agent responses.

O'REILLY®

Message Queues for Agent Workloads

- **Mechanism:** An asynchronous buffer where tasks wait until a worker is available to process them.
- **Decoupling:** Separates rapid user input from slow, expensive agent execution.
- **Backpressure:** Absorbs traffic spikes so workers don't crash from overload.
- **Cost / Request Limit Control:** Worker count limits max concurrent requests to external APIs, regardless of queue depth.
- **Priority Lanes:** Separate queues allow paid users to bypass free-tier congestion.
- **Dead Letter Queues:** capturing failed tasks prevents retries from blocking the system.

O'REILLY®

State Store Scaling

- **Hot Storage (Redis):** Ephemeral context/history. Cluster mode required for horizontal scale.
- **Warm Storage (SQL/NoSQL):** Durable user data. Offload heavy reads to Replicas.
- **Cold Storage (S3):** Offload large artifacts (images, PDFs) to keep DBs lean.
- **The Bottleneck:** Workers exhaust DB connections fast. Connection Pooling (PgBouncer) is mandatory.
- **Context Management:** Implement rolling windows or summarization to prevent unbounded growth.
- **Lifecycle:** Use TTLs (Time-To-Live) to auto-clean abandoned session state.

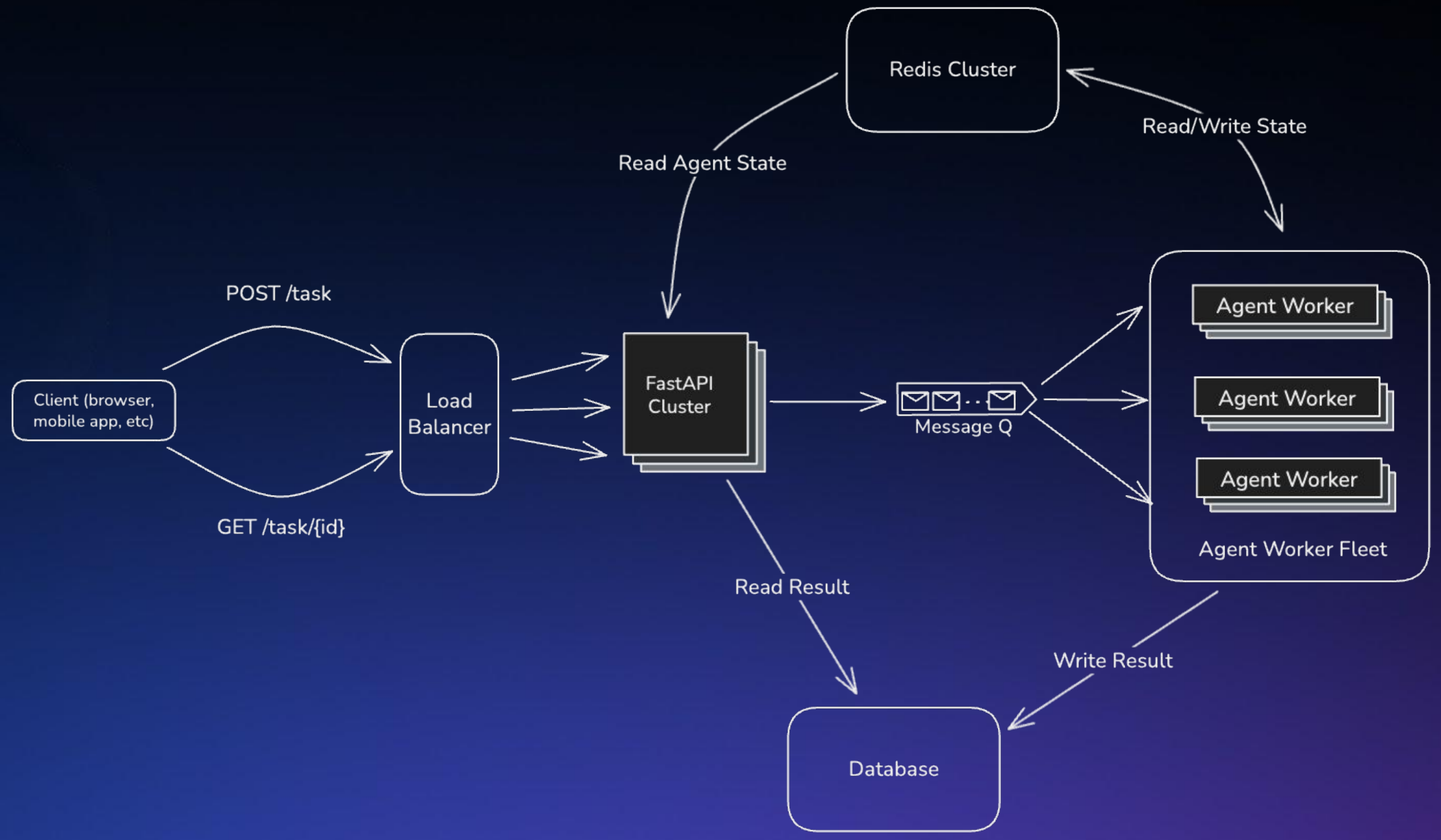
O'REILLY®

Cost Control & Fairness

- **The Risk:** LLM costs scale linearly; one runaway loop can drain budgets instantly.
- **Token Budgets:** Enforce hard caps per tenant/user per time window (e.g., $10/day).
- **Concurrency Limits:** Restrict parallel tasks per tenant to prevent resource monopolization.
- **Noisy Neighbors:** Use fair-scheduling (round-robin) so heavy users don't block others.
- **Throttling:** Return 429 Too Many Requests or queue position rather than over-scaling.
- **Tiering:** Reserve high-throughput infrastructure/limits for premium tiers only.

O'REILLY

Example: Scaled Architecture



A Horizontally Scaled Architecture for Agentic Systems

O'REILLY®

Scaling Best Practices

- **Assume Failure:** LLMs time out, tools break. Design for automatic retries.
- **Graceful Shutdowns:** Nodes must finish current tasks before terminating (don't kill "thinking" agents).
- **Scale on Metrics:** Monitor queue latency and token burn rate, not just CPU.
- **Externalize Everything:** Logs, state, and secrets must live outside the container.
- **Control Capacity:** More workers = higher cost. Use queues to buffer; scale carefully.
- **Incremental Rollout:** Test scaling behavior (and costs) under synthetic load before production.

O'REILLY®

4.16 Implementing Cost Optimization Strategies

O'REILLY®

Understanding Cost Drivers in AI Agents

Primary Cost Components

- **LLM API calls:** 70-80% of total costs
- **Tool execution:** Web searches, API calls, database queries
- **Infrastructure:** Compute, storage, bandwidth
- **Embeddings:** Vector generation for retrieval and caching

O'REILLY®

Understanding Cost Drivers in AI Agents

Cost Impact Factors

- Model selection can have a massive impact on overall costs
- **Token usage:** Input and output tokens billed separately
- **Request frequency:** Number of LLM calls per workflow
- **Tool complexity:** External API costs and latency

O'REILLY®

Intelligent Model Routing

Strategy: Match Model Capability to Task Complexity

- Simple queries → Lightweight models (GPT-4o/mini, Claude Haiku)
- Complex reasoning → Premium models (GPT-5, Claude Sonnet/Opus)
- Use appropriate model to classify query complexity first
- Route based on confidence scores and task requirements

Cost Impact

- **Potential savings**: 40-60% on average workloads
- **Trade-off**: Classification step adds minimal latency
- **Best for**: High-volume applications with mixed complexity

O'REILLY®

Semantic Caching

Strategy: Reuse Responses for Similar Queries

- **Traditional caching:** Exact string match only
- **Semantic caching:** Match queries by meaning (embeddings)
- **Similarity threshold:** 0.90-0.95 cosine similarity
- **Time-to-live (TTL):** Balance freshness vs cost savings

How It Works

- Convert query to embedding vector
- Compare with cached query embeddings
- Return cached response if similarity exceeds threshold
- Store new query-response pairs in cache

O'REILLY®

Semantic Caching

Cost-Benefit Analysis

- **Cache hit rate:** 10-30% typical; up to 50% in narrow, repetitive domains
- **Savings per hit:** ~99% (embedding + vector lookup vs LLM call)
  - **Embedding cost:** ~$0.0001 vs $0.01-0.03 for LLM call
  - **Vector DB query cost:** ~$0.0001-0.001 per lookup (often overlooked)
- **Key risk:** Semantic similarity ≠ identical intent; false positives return wrong answers

O'REILLY®

Tool Result Reuse

Strategy: Reuse Responses for Similar Queries

- Web search results (highest cost and latency)
- Database query results (for frequently accessed data)
- External API responses (when data changes infrequently)
- Document retrieval results (for static knowledge bases)

O'REILLY

Tool Result Reuse

Cache Configuration

- TTL based on data freshness requirements
  - Real-time data: 5-15 minutes
  - Semi-static data: 1-6 hours
  - Static reference data: 24+ hours
- **Cache key**: Hash of tool name + parameters
- **Storage**: High speed kv stores like Redis

Impact Metrics

- **Web search**: $0.001-0.01 per query saved
- **Latency reduction**: 500-2000ms per cache hit
- **Combined savings**: Cost + improved user experience

O'REILLY®

Prompt Optimization for Token Usage

Strategy: Minimize Token Usage Without Losing Quality

- Remove verbose instructions and examples
- Use structured formats (JSON, bullet points)
- Compress system prompts with key phrases
- Limit conversation history to relevant context

O'REILLY®

Common Pitfall: Caching without Invalidation

Semantic cache hit rates look great until you realize users are getting stale information.

A "latest news" query may return last month's news.

Implement TTL and invalidation strategies appropriate to your data freshness requirements.

O'REILLY®

4.17 Building Deep Agents for Complex Tasks

O'REILLY®

Patterns for Designing Sub-Agents

- **Pattern 1: Functional Specialization**
  - Divide by task type: research, analysis, writing, coding
  - Example: Researcher → Analyst → Writer
- **Pattern 2: Domain Specialization**
  - Divide by knowledge domain: finance, healthcare, legal
  - Example: Financial analyst → Medical researcher → Legal reviewer
- **Pattern 3: Process Stage Specialization**
  - Divide by pipeline stage: ingest, process, validate, output
  - Example: Data collector → Cleaner → Analyzer
- **Pattern 4: Capability-Based Specialization**
  - Divide by technical capability: API access, database, file operations
  - Example: Web scraper → Database writer → Report generator

O'REILLY®

The Orchestrator / Supervisor Pattern Dominates

- Used by Claude Code, LangGraph Deep Agents, and production systems
- Maximizes parallelization for independent tasks
- Clear separation: planning vs. execution
- Simpler to reason about than complex choreography

O'REILLY®

Division of Responsibilities

- **Orchestrator / Supervisor**
  - Analyzes overall tasks
  - Decomposes into sub-tasks
  - Routes to workers
  - Synthesizes final response
- **Worker / Sub-Agents**
  - Execute specific sub-tasks
  - Use specialized tools
  - Return structured results

O'REILLY®

Designing Effective Sub-agent Boundaries

Good Boundaries

- Distinct toolsets
- Natural handoff points
- Expertise separation
- Independent testing
- Reduces overall prompt complexity


O'REILLY®